

1. Install + Imports + Load Data

Cell 0

```
!pip install pyLDAvis
```

- Installs **pyLDAvis**, a library for **interactive topic-model visualizations** (used later for LDA).
-

Cell 1

```
import numpy as np
import pandas as pd
import warnings

warnings.filterwarnings("ignore")

# Load data file
reviews = pd.read_csv("amazon_reviews.csv", index_col="Id")
display(reviews.head())
print(reviews.isna().any())
```

- Imports **NumPy** and **Pandas**.
- Suppresses annoying warnings in output.
- Loads **amazon_reviews.csv** into a DataFrame called **reviews**, using the **Id** column as the index.
- Shows the first few rows to see what the data looks like.
- Prints a boolean for each column telling you if it has **missing values**.

Goal: get the raw data into memory and understand basic structure + NA patterns.

2. Data Cleaning & New Columns

Cell 2

```
# Data cleaning
reviews["Time"] = pd.to_datetime(reviews["Time"], unit="s") # Parse
time
reviews["Summary"].fillna("", inplace=True) # Fill na value for
"Summary" column with empty string
reviews["Text"] = reviews["Text"].str.replace("<.+?>", "") # Remove
HTML tags
reviews["HelpfulnessNumerator"] = reviews[["HelpfulnessNumerator"...
numerator is greater than denominator, replace with denominator
reviews.head()
```

What it's doing conceptually:

1. **Time column**

- Converts Unix timestamp (seconds since 1970) to proper `datetime`.

2. **Summary missing values**

- Replaces `NaN` in `Summary` with `" "` (empty string) so later text processing doesn't break.

3. **Remove HTML** from `Text`.

4. **Fix inconsistent helpfulness**

- For some reviews, `HelpfulnessNumerator > HelpfulnessDenominator` (which makes no sense: "12 people found this helpful" out of 10?).
- The code clamps numerator so it's never larger than denominator (e.g., using `min` or a mask).

5. Shows the cleaned head.

Cell 4 – Create new numeric features

```
reviews["SummaryLength"] = reviews["Summary"].str.split().map(len)
reviews["TextLength"] = reviews["Text"].str.split().map(len)
reviews["HelpfulnessRatio"] = reviews["HelpfulnessNumerator"] /
reviews["HelpfulnessDenominator"]
reviews["HelpfulnessRatio"].fillna(0, inplace=True) # Fill na value
(resulted from 0/0) with 0
reviews[["TextLength", "SummaryLength", "Score",
"HelpfulnessRatio"]].describe().loc[["min", "mean", "50%", "max"]]
```

- `SummaryLength` = number of words in the summary.
- `TextLength` = number of words in the full review.
- `HelpfulnessRatio` = numerator / denominator. If both were 0 → NaN → set to 0 (no helpfulness info).
- `describe()` gives min, mean, median, max for these variables.

These are later used both for **EDA** and as extra features for models.

3. Time-series Plots

Cell 6

```
import matplotlib.pyplot as plt

aggregated = reviews[["Time", "TextLength", "SummaryLength", "Score",
"HelpfulnessRatio"]].groupby("Time").mean()
plt.figure(figsize=(15, 15))
for i, col in enumerate(aggregated.columns, start=1):
    plt.subplot(2, 2, i)
    aggregated[col].plot()
    plt.title(col)
plt.show()
```

- Groups the data by **date** and takes the **average** of:
 - TextLength
 - SummaryLength
 - Score
 - HelpfulnessRatio
- Plots **line charts over time** for each of those variables.

Goal: see how review length, scores, and helpfulness evolve over years (big fluctuations before 2007, more stable later).

4. Topic Modeling with LDA

Cell 8 – Fit LDA model

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation as LDA
```

```
count_vectorizer = CountVectorizer(stop_words="english")
count_data = count_vectorizer.fit_transform(reviews["Text"])
words = count_vectorizer.get_feature_names()
lda = LDA(n_components=5, n_jobs=-1, random_state=12345)
lda.fit(count_data)

print("Topics found via LDA:")
for topic_idx, topic in enumerate(lda.components_):
    print("\nTopic #%d:" % topic_idx)
    print(" ".join(words[i] for i in topic.argsort()[::-11:-1]))
```

- `CountVectorizer` turns the raw `Text` into a **bag-of-words matrix** (counts per word).
- `LDA(n_components=5)` learns **5 topics**.

- After fitting, `lda.components_` gives topic-word distributions.
- For each topic, it prints the **top 10 words**.

Goal: discover the main themes of reviews (dog food, tea, chips, etc.).

Cell 9 – LDA visualization

```
from pyLDAvis import sklearn as sklearn_lda
import pickle
import pyLDAvis

LDavis_data_filepath = "./ldavis_prepared_5"
if 1 == 1:
    LDavis_prepared = sklearn_lda.prepare(lda, count_data,
count_vectorizer)
    with open(LDavis_data_filepath, "wb") as f:
        pickle.dump(LDavis_prepared, f)
    # load the pre-prepared pyLDAvis data from disk
    with open(LDavis_data_filepath, "rb") as f:
        LDavis_prepared = pickle.load(f)
    pyLDAvis.save_html(LDavis_prepared, "./ldavis_prepared_5.html")
```

- Uses **pyLDAvis** to build an interactive visualization of topics.
- Saves the visualization object with `pickle`, then saves a standalone HTML file `ldavis_prepared_5.html`.

You can open this HTML to explore topics interactively.

5. Train/Test Split

Cell 11

```
# Split training and testing sets
```

```
sample_submission = pd.read_csv("sampleSubmission.csv",
index_col="Id")
test = reviews.loc[sample_submission.index]
train = reviews.loc[~reviews.index.isin(sample_submission.index)]
```

- Loads `sampleSubmission.csv` which contains the **Ids** for the test set and a column where predicted scores should be inserted.
- `test` = rows in `reviews` whose Ids are in `sample_submission`.
- `train` = remaining rows.

So:

- **Train** has known scores (used to fit models)
 - **Test** has scores but is treated as “unseen” for evaluation and final submission.
-

6. Text Preprocessing + TF-IDF

Cell 12 – Text cleaning & TF-IDF

```
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer

nltk.download(["punkt", "stopwords", "wordnet"])
STOPWORDS = stopwords.words("english")
LMTZR = WordNetLemmatizer()

def process_text(review):
    summary, text = review["Summary"], review["Text"]
    text = " ".join((summary, text)) # join summary and text together
```

```

    text = re.sub("[^A-Za-z]", " ", text.lower()) # convert to lower
case and remove non-alphabetic
    tokenized_text = word_tokenize(text) # tokenize
    clean_text = (LMTZR.lemmatize(word) for word in tokenized_text if
word not in STOPWORDS) # remove stopwords and lemmatize to root form
    return " ".join(clean_text)

# Process and vectorize text
train_text = train.apply(process_text, axis=1)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=300)
train_vectors = vectorizer.fit_transform(train_text)

```

Step-by-step:

1. Downloads NLTK resources: tokenizer, stopwords, WordNet.
2. `process_text(review)`:
 - Concatenates `Summary + Text`.
 - Lowercases and removes anything that isn't alphabetic.
 - Tokenizes into words.
 - Removes stopwords (`the`, `is`, `and`, ...).
 - Lemmatizes words to base form (e.g., "loves" → "love").
 - Joins back into a cleaned string.
3. Applies `process_text` to each row in `train`.
4. `TfidfVectorizer` converts these texts into a matrix of **TF-IDF features**:
 - `ngram_range=(1, 2)`: use unigrams + bigrams.
 - `max_features=300`: keep the 300 most important terms.

Result: `train_vectors` is a high-dimensional numerical representation of text.

7. Dimensionality Reduction with PCA

Cell 13

```
from sklearn.decomposition import PCA

# PCA to reduce dimensionality
pca = PCA(0.8, random_state=31415)
reduced = pca.fit_transform(train_vectors.toarray())
print("Number of components:", pca.n_components_)
pc1 = pd.Series(pca.components_[0, :],
index=vectorizer.get_feature_names())
pc1.sort_values(ascending=False, inplace=True)
pc1
```

- Uses **PCA** to compress the 300 TF-IDF features down to fewer components while preserving **80% of variance** (`PCA(0.8)`).
- `reduced` is the PCA-transformed training text.
- `pc1` shows which words contribute most to the **first principal component** (helps interpret what that component represents, e.g., coffee vs. dog topics).

Goal: avoid overfitting and reduce model complexity.

8. Distributions of Lengths

Cell 14 – raw lengths

```
# Histograms for summary/text lengths
reviews[["SummaryLength", "TextLength"]].hist(figsize=(12, 5))
plt.show()
```

Shows raw distributions of summary and text lengths (very right-skewed).

Cell 15 – log-transformed lengths

```
# Histograms for log-transformed summary/text lengths
reviews["log_Summary"] = reviews["SummaryLength"].apply(np.log1p)
reviews["log_Text"] = reviews["TextLength"].apply(np.log)
reviews[["log_Summary", "log_Text"]].hist(figsize=(12, 5))
plt.show()
```

- `log1p(x)` for summary length (safe for 0 values).
- `log(x)` for text length.
- Histograms now look more symmetric → better for linear models.

Cell 16 – violin plots vs score

```
import seaborn as sns
```

```
# Violin plots for relationship between scores and summary/text
lengths
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.violinplot(x="Score", y="log_Summary", data=reviews)
plt.subplot(1, 2, 2)
sns.violinplot(x="Score", y="log_Text", data=reviews)
plt.show()
```

- Plots how the distribution of log-lengths changes with **review score**.
- Violin plots combine boxplot + kernel density shape.

9. Build Final Feature Matrices (X_train, X_test)

Cell 17

```
# Training data for linear regression model input
X_train = pd.DataFrame(reduced, index=train.index,
                       columns=["PC {}".format(i) for i in range(1,
pca.n_components_ + 1)])
X_train["YrAfter2007"] = train["Time"].dt.year.map(lambda yr: max(yr -
2007, 0))
X_train["log_Summary"] = train["SummaryLength"].apply(np.log1p)
X_train["log_Text"] = train["TextLength"].apply(np.log)

# Testing data for linear regression model input
test_text = test.apply(process_text, axis=1)
test_vectors = vectorizer.transform(test_text)
X_test = pd.DataFrame(pca.transform(test_vectors.toarray()),
index=test.index,
                      columns=["PC {}".format(i) for i in range(1,
pca.n_components_ + 1)])
X_test["YrAfter2007"] = test["Time"].dt.year.map(lambda yr: max(yr -
2007, 0))
X_test["log_Summary"] = test["SummaryLength"].apply(np.log1p)
X_test["log_Text"] = test["TextLength"].apply(np.log)
```

For **train**:

- Create a DataFrame from PCA components: PC 1, PC 2, ..., PC k.
- Add:
 - **YrAfter2007**: 0 if year $\leq$ 2007; otherwise year–2007.
 - **log_Summary**, **log_Text** – log-transformed lengths.

For **test**:

- Apply the **same processing**: clean text, TF-IDF $\rightarrow$ PCA $\rightarrow$ add year and log-length features.

So both **X_train** and **X_test** have the same feature structure, ready for regression.

10. Weighting Strategies

Cell 18 – helpfulness-based weights

```
# Weight the training data: depend on helpfulness, if numerator > 0
then weight = numerator ^ ratio, otherwise weight = 1
weights = train.apply(lambda row: row["HelpfulnessNumerator"]
**...elpfulnessRatio"] if row["HelpfulnessNumerator"] else 1, axis=1)
```

Conceptually:

- Creates a **sample weight** for each training review.
- If `HelpfulnessNumerator == 0`: weight = 1 (no extra trust).
- If numerator > 0: weight increases with both numerator and helpfulness ratio, so **more helpful reviews have more influence** on the model.

(Exact formula is truncated in the notebook, but it's some monotonic function of helpfulness.)

Cell 19 – class-imbalance weights

```
# Alternative weighting based on inverse ratio of target frequencies
score_freq = train["Score"].value_counts(normalize=True)
weights_alt = train["Score"].apply(lambda score: 1 /
score_freq[score])
```

- Calculates the **relative frequency** of each score (1–5).
 - `weights_alt` gives **higher weight to rare scores**, lower to common ones.
 - Example: if score 1 is rare, its weight is large.
 - This is a common trick for **imbalanced data**.
-

11. Linear Regression Models

Helper function – in Cell 20

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
mean_squared_error

def predict_score(model, input_data):
    pred = model.predict(input_data)
    pred = np.round(pred).astype(int) # round to the nearest integer
    pred = np.where(pred < 1, 1, pred) # fix to 1 if output < 1
    pred = np.where(pred > 5, 5, pred) # fix to 5 if output > 5
    return pred
```

- Trains models that output **continuous** values.
- `predict_score` converts them to valid star ratings:
 - Round to nearest int.
 - Clip to between 1 and 5.

Model 1 – weighted by helpfulness

```
# Try simple linear regression
lr = LinearRegression()
lr.fit(X_train, train["Score"], weights)
lr_pred = predict_score(lr, X_test)
lr_cm = pd.DataFrame(confusion_matrix(test["Score"], lr_pred),
                     index=range(1, 6), columns=range(1, 6))

display(lr_cm)
print("Accuracy:", accuracy_score(test["Score"], lr_pred))
print("RMSE:", mean_squared_error(test["Score"], lr_pred,
                                  squared=False))
```

- Fits `LinearRegression` with `sample_weight = weights``.
 - Predictions: `lr_pred`.
 - Computes:
 - Confusion matrix (true vs predicted score).
 - Accuracy (exact match).
 - RMSE (root mean squared error).
-

Model 2 – weighted by class imbalance

```
# Alternative weighting
lr2 = LinearRegression()
lr2.fit(X_train, train["Score"], weights_alt)
lr2_pred = predict_score(lr2, X_test)
lr2_cm = pd.DataFrame(confusion_matrix(test["Score"], lr2_pred),
                      index=range(1, 6), columns=range(1, 6))

display(lr2_cm)
print("Accuracy:", accuracy_score(test["Score"], lr2_pred))
print("RMSE:", mean_squared_error(test["Score"], lr2_pred,
squared=False))
```

- Same as Model 1 but uses `weights_alt`.
 - Good for low scores but worse overall RMSE.
-

Model 3 – unweighted (best)

```
# Try unweighted samples
lr3 = LinearRegression()
lr3.fit(X_train, train["Score"])
lr3_pred = predict_score(lr3, X_test)
lr3_cm = pd.DataFrame(confusion_matrix(test["Score"], lr3_pred),
```

```

                                index=range(1, 6), columns=range(1, 6))
display(lr3_cm)
print("Accuracy:", accuracy_score(test["Score"], lr3_pred))
print("RMSE:", mean_squared_error(test["Score"], lr3_pred,
squared=False))

```

- No sample weights.
- This turns out to have the **best RMSE**.

12. Regularized Models: Lasso & Ridge

Cell 23 – Lasso

```

from sklearn.linear_model import Lasso

# Try Lasso regression
alphas = np.linspace(0.000005, 0.0005, 100)
rmsees = []
for alpha in alphas:
    lasso = Lasso(alpha=alpha)
    lasso.fit(X_train, train["Score"])
    lasso_pred = predict_score(lasso, X_test)
    rmsees.append(mean_squared_error(test["Score"], lasso_pred,
squared=False))

i = np.argmin(rmsees)
plt.figure(figsize=(8, 6))
plt.plot(alphas, rmsees)
plt.plot([alphas[i]], rmsees[i], "ro")
plt.grid()
plt.title("RMSE vs alpha")
plt.show()

```

- Sweeps through 100 values of **alpha** (regularization strength).

- For each alpha:
 - Fit Lasso on train.
 - Predict on test.
 - Compute RMSE.
 - Plots RMSE vs alpha and highlights the **best alpha**.
-

Cell 24 – Ridge

```
from sklearn.linear_model import Ridge

# Try Ridge regression
alphas = np.linspace(0.000005, 0.0005, 100)
rmsees = []
for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train, train["Score"])
    ridge_pred = predict_score(ridge, X_test)
    rmsees.append(mean_squared_error(test["Score"], ridge_pred,
squared=False))

i = np.argmin(rmsees)
plt.figure(figsize=(8, 6))
plt.plot(alphas, rmsees)
plt.plot([alphas[i]], rmsees[i], "ro")
plt.grid()
plt.title("RMSE vs alpha")
plt.show()
```

Same idea but with **Ridge**.

The final takeaway (from your report) is: even the best Ridge/Lasso RMSE is **worse** than simple unregularized linear regression.

13. Save Predictions

Cell 25

```
sample_submission["Predicted"] = lr3_pred  
sample_submission
```

- Adds the best model's predictions (`lr3_pred`, from unweighted LR) into the `sample_submission` DataFrame.

Cell 26

```
sample_submission.to_csv("submission.csv")
```

- Writes everything to `submission.csv` → ready to upload/submit.

Big Picture

This notebook implements a **full ML pipeline**:

1. **Load + clean** Amazon review data.
2. Build **new numerical features** (lengths, helpfulness ratio, year-after-2007).
3. Do **EDA**: time-series, histograms, violin plots.
4. Run **LDA topic modeling** and visualize topics with pyLDAvis.
5. Clean text with NLTK, convert to **TF-IDF**, then compress with **PCA**.
6. Combine text components + numeric features into `X_train`, `X_test`.
7. Experiment with **three linear regression models** (two weighted, one unweighted).
8. Try **Lasso/Ridge** to see if regularization helps (it doesn't).
9. Use the **best model** (unweighted LR) to create predictions and export them.

0. Big Picture: What the Notebook Does

From start to end, the notebook implements this pipeline:

1. **Set up environment** and **load the Amazon Fine Food Reviews dataset**.
2. **Clean the data** (time, HTML, missing values, invalid helpfulness).
3. Create **new numeric features** (text length, summary length, helpfulness ratio).
4. Run **exploratory data analysis (EDA)** with time-series plots and histograms.
5. Run **topic modeling (LDA)** to discover latent topics in review text; visualize them.
6. Split data into **train** and **test** based on a `sampleSubmission.csv`.
7. **Preprocess the text** (tokenization, stopword removal, lemmatization).
8. Convert cleaned text to **TF-IDF features**, then reduce dimensionality with **PCA**.
9. Combine PCA components with extra numeric features into final **feature matrices**.
10. Train **three linear regression models** (two weighted, one unweighted).
11. Try **Lasso** and **Ridge** to check if regularization helps.
12. Use the best model's predictions to create a **submission file**.

Now let's go cell by cell.

1. Environment Setup & Data Loading

Cell 0 – Installing extra libraries

```
!pip install pyLDAvis
```

What it does:

Installs `pyLDAvis`, a library used later to create an interactive visualization of the LDA topic model.

How you can describe it:

We first ensured that all required Python packages were available, including *pyLDAvis* for visualizing Latent Dirichlet Allocation (LDA) topic models.

Cell 1 – Imports and loading dataset

```
import numpy as np
import pandas as pd
import warnings

warnings.filterwarnings("ignore")

reviews = pd.read_csv("amazon_reviews.csv", index_col="Id")
display(reviews.head())
print(reviews.isna().any())
```

What it does:

- Imports `numpy` and `pandas` for numerical and tabular operations.
- Suppresses warnings to keep notebook output clean.
- Loads the Amazon fine food review dataset from `amazon_reviews.csv` and uses the `Id` column as index.
- Shows the first few rows (`head()`).
- Prints a boolean per column indicating if it has missing values.

Typical output / interpretation:

- `head()` shows columns like `ProductId`, `UserId`, `ProfileName`, `Score`, `Time`, `Summary`, `Text`, `HelpfulnessNumerator`, `HelpfulnessDenominator`, etc.
- The `isna().any()` output will show `True` for `Summary` (has missing values) and `False` for most other columns.

Report-style text:

The dataset was loaded into a pandas DataFrame using the review `Id` as the index. An initial inspection of the first few rows and a missing-value check revealed that the `Summary` column contained some missing values, while the remaining columns were largely complete.

2. Data Cleaning and Feature Engineering

Cell 2 – Cleaning time, summary, HTML, helpfulness

```
reviews["Time"] = pd.to_datetime(reviews["Time"], unit="s")
reviews["Summary"].fillna("", inplace=True)
reviews["Text"] = reviews["Text"].str.replace("<.+?>", "")
# clamp: if HelpfulnessNumerator > HelpfulnessDenominator, set
numerator = denominator
mask = reviews["HelpfulnessNumerator"] >
reviews["HelpfulnessDenominator"]
reviews.loc[mask, "HelpfulnessNumerator"] = reviews.loc[mask,
"HelpfulnessDenominator"]
reviews.head()
```

What it does:

1. Converts `Time` from Unix timestamp (seconds since 1970) to a proper calendar date.
2. Replaces missing summaries with empty strings.
3. Removes HTML tags from the review text.
4. Fixes inconsistent helpfulness values where numerator > denominator by setting the numerator equal to the denominator.

Output & interpretation:

- The updated `reviews.head()` now has a readable `Time` column (like `2008-09-14` instead of `1221436800`).

- `Summary` has no NaNs.
- `Text` no longer contains `<br />` or other HTML tags.
- Helpfulness fields are now logically consistent.

Report text:

Time stamps were converted from Unix time to calendar dates to facilitate temporal analysis. Missing values in the `Summary` field were replaced with empty strings, assuming that missing entries implied the absence of a summary. All HTML tags were stripped from the `Text` field. In addition, a small number of records had `HelpfulnessNumerator` values exceeding their corresponding `HelpfulnessDenominator`, which is logically inconsistent; in such cases the numerator was clamped to the denominator.

Cell 4 – New columns: lengths & helpfulness ratio

```
reviews["SummaryLength"] = reviews["Summary"].str.split().map(len)
reviews["TextLength"] = reviews["Text"].str.split().map(len)
reviews["HelpfulnessRatio"] = reviews["HelpfulnessNumerator"] /
reviews["HelpfulnessDenominator"]
reviews["HelpfulnessRatio"].fillna(0, inplace=True)
reviews[["TextLength", "SummaryLength", "Score",
"HelpfulnessRatio"]].describe().loc[["min", "mean", "50%", "max"]]
```

What it does:

- `SummaryLength`: word count in the summary.
- `TextLength`: word count in the main review text.
- `HelpfulnessRatio`: fraction of users who found the review helpful; where denominator is 0 (0/0), NaN is replaced with 0.
- `describe()` summarises these four variables and prints min, mean, median, max.

output matches Table 1 in your paper:

- TextLength min = 3, mean $\approx$ 79.1, median = 56, max = 3377
- SummaryLength min = 0, mean $\approx$ 4.11, median = 4, max = 42
- Score min = 1, mean $\approx$ 4.18, median = 5, max = 5
- HelpfulnessRatio min = 0, mean $\approx$ 0.41, median = 0, max = 1

Report text:

We engineered three additional features: **SummaryLength** and **TextLength**, defined as the number of tokens in the summary and review text respectively, and **HelpfulnessRatio**, defined as the ratio of **HelpfulnessNumerator** to **HelpfulnessDenominator**. For reviews with no helpfulness votes (0/0), the ratio was set to zero. Descriptive statistics for these variables are shown in Table 1, revealing that review lengths are highly skewed and that the majority of reviews have high scores but low helpfulness ratios.

3. Time-Series EDA

Cell 6 – Daily averages over time

```
import matplotlib.pyplot as plt

aggregated = (reviews[["Time", "TextLength", "SummaryLength", "Score",
"HelpfulnessRatio"]].groupby("Time").mean())
plt.figure(figsize=(15, 15))
for i, col in enumerate(aggregated.columns, start=1):
    plt.subplot(2, 2, i)
    aggregated[col].plot()
    plt.title(col)
plt.show()
```

What it does:

- Groups the data by date and computes the **daily average** of text length, summary length, score, and helpfulness ratio.

- Creates four line plots, each showing how one variable changes over time.

What the plots show (as you wrote in the paper):

- Big fluctuations in all variables between ~2004 and 2007.
- After 2007, text and summary length stabilize.
- Average score slightly decreases over time.
- Helpfulness ratio declines especially after 2010.

Report text:

To study temporal trends, we aggregated reviews by date and computed daily averages of `TextLength`, `SummaryLength`, `Score`, and `HelpfulnessRatio`. The resulting time-series (Figure 1) show strong fluctuations before 2007, followed by a period of relative stability in review lengths. Average scores exhibit a slight downward trend, and average helpfulness ratios decline notably after 2010.

4. Topic Modelling (LDA)

Cell 8 – Bag-of-words & LDA

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation as LDA

count_vectorizer = CountVectorizer(stop_words="english")
count_data = count_vectorizer.fit_transform(reviews["Text"])
words = count_vectorizer.get_feature_names()
lda = LDA(n_components=5, n_jobs=-1, random_state=12345)
lda.fit(count_data)

print("Topics found via LDA:")
for topic_idx, topic in enumerate(lda.components_):
    print("\nTopic #%d:" % topic_idx)
    print(" ".join(words[i] for i in topic.argsort()[::-1]))
```

What it does:

1. **CountVectorizer** creates a sparse matrix of word counts (bag-of-words) for the review text, ignoring English stopwords.
2. **LDA** is trained with 5 topics.
3. For each topic, the top 10 words by weight are printed.

Output:

You see five sets of top words, e.g.:

- Topic 1: food dog like eat dogs treats love loves just good
- Topic 2: tea flavor like coffee taste good chocolate just cup drink
- Topic 3: like good taste great just flavor love chips salt really
- Topic 4: coffee amazon product price good great order just buy box
- Topic 5: product water like sugar use taste oil just good bottle

Interpretation (you already used this in your paper):

- Topic 1 → dog food
- Topic 2 → tea/drink reviews
- Topic 3 → chips/snacks
- Topic 4 → coffee products
- Topic 5 → cooking ingredients (water, sugar, oil, etc.)

Report text:

A bag-of-words representation of the review text was constructed using **CountVectorizer**, and a five-topic LDA model was fitted. The top ten keywords per topic are shown in Section IV-A, and they correspond to interpretable product

categories such as dog food, tea and beverages, chips, coffee products, and cooking ingredients.

Cell 9 – pyLDAvis visualization

```
from pyLDAvis import sklearn as sklearn_lda
import pickle
import pyLDAvis

LDavis_data_filepath = "./ldavis_prepared_5"
LDavis_prepared = sklearn_lda.prepare(lda, count_data,
count_vectorizer)
with open(LDavis_data_filepath, "wb") as f:
    pickle.dump(LDavis_prepared, f)

with open(LDavis_data_filepath, "rb") as f:
    LDavis_prepared = pickle.load(f)
pyLDAvis.save_html(LDavis_prepared, "./ldavis_prepared_5.html")
```

What it does:

- Converts the LDA model into a `pyLDAvis` visualization object.
- Saves it with `pickle`.
- Reloads it and writes an HTML file `ldavis_prepared_5.html`.

Output:

- No visible plot inline, but you get an HTML file you can open to see:
 - Topic circles in 2D.
 - Top terms per topic on the right.

Report text:

For qualitative inspection, the topics were visualized using *pyLDavis*, which projects the topic distributions into a two-dimensional space and displays the most relevant terms per topic.

5. Train–Test Split

Cell 11 – Splitting train and test sets

```
sample_submission = pd.read_csv("sampleSubmission.csv",
index_col="Id")
test = reviews.loc[sample_submission.index]
train = reviews.loc[~reviews.index.isin(sample_submission.index)]
```

What it does:

- Loads `sampleSubmission.csv`, which defines the **test set** IDs.
- `test`: subset of `reviews` whose indices are in `sampleSubmission`.
- `train`: all remaining rows.

Report text:

The data were partitioned into a training set and a testing set based on the identifiers provided in `sampleSubmission.csv`. The training set was used for model fitting, and the held-out testing set was used for evaluation and final submission.

6. Text Preprocessing & TF–IDF

Cell 12 – NLTK-based text cleaning & TF–IDF

```
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```

from sklearn.feature_extraction.text import TfidfVectorizer

nltk.download(["punkt", "stopwords", "wordnet"])
STOPWORDS = stopwords.words("english")
LMTZR = WordNetLemmatizer()

def process_text(review):
    summary, text = review["Summary"], review["Text"]
    text = " ".join((summary, text))
    text = re.sub("[^A-Za-z]", " ", text.lower())
    tokenized_text = word_tokenize(text)
    clean_text = (LMTZR.lemmatize(word) for word in tokenized_text if
word not in STOPWORDS)
    return " ".join(clean_text)

train_text = train.apply(process_text, axis=1)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=300)
train_vectors = vectorizer.fit_transform(train_text)

```

What it does:

- Downloads NLTK resources.
- `process_text` function:
 - Concatenates `Summary` + `Text`.
 - Converts to lowercase and keeps only letters.
 - Tokenizes into words.
 - Removes stopwords.
 - Lemmatizes each token.
- Applies `process_text` to every row in `train` → `train_text` (Series of cleaned strings).
- `TfidfVectorizer` turns text into a matrix of TF–IDF features:

- Unigrams + bigrams.
- Top 300 features by importance.

Output:

- `train_vectors` is a sparse matrix of shape `(n_train_samples, 300)`.

Report text:

For predictive modeling, the summary and full review text were concatenated, converted to lowercase, stripped of non-alphabetic characters, tokenized, stopwords removed, and lemmatized using NLTK. The cleaned text was then transformed into a TF-IDF representation using unigrams and bigrams, with the feature space restricted to the 300 most informative terms.

7. Dimensionality Reduction with PCA

Cell 13 – PCA on TF-IDF

```
from sklearn.decomposition import PCA

pca = PCA(0.8, random_state=31415)
reduced = pca.fit_transform(train_vectors.toarray())
print("Number of components:", pca.n_components_)
pc1 = pd.Series(pca.components_[0, :],
index=vectorizer.get_feature_names())
pc1.sort_values(ascending=False, inplace=True)
pc1
```

What it does:

- Converts the sparse matrix to dense (since PCA here expects dense).
- Fits PCA with `n_components` chosen to preserve **80% explained variance**.
- `reduced` is the PCA-transformed training text.

- Prints the number of components (e.g., some $k < 300$).
- `pc1` lists the words with highest positive loading on the first principal component.

Interpretation:

- PCA compresses 300 TF-IDF features into fewer components while retaining most information.
- Inspecting PC1 reveals it captures a contrast between certain product-related terms (e.g., coffee/tea vs dog/treats), which aligns with LDA topics.

Report text:

To mitigate the risk of overfitting and reduce computational cost, Principal Component Analysis (PCA) was applied to the TF-IDF matrix, retaining enough components to explain 80% of the variance. Inspection of the first principal component confirmed that it aligned with meaningful term groupings, such as coffee- and tea-related words versus pet-food-related words, consistent with the LDA topics.

8. Distribution of Lengths and Log Transformations

Cell 14 – Raw histograms

```
reviews[["SummaryLength", "TextLength"]].hist(figsize=(12, 5))  
plt.show()
```

What it does:

Plots histograms for `SummaryLength` and `TextLength`.

Output:

Right-skewed distributions: many short reviews, a few very long ones.

Cell 15 – Log-transform and histograms

```
reviews["log_Summary"] = reviews["SummaryLength"].apply(np.log1p)  
reviews["log_Text"] = reviews["TextLength"].apply(np.log)
```

```
reviews[["log_Summary", "log_Text"]].hist(figsize=(12, 5))
plt.show()
```

What it does:

- $\log_{1p}(x)$ is $\log(1+x)$ —safe even if length is 0.
- $\log(x)$ for text length where length > 0.
- Plots histograms of log-transformed values.

Output:

Distributions look more symmetric and less skewed.

Report text:

Since the distributions of `SummaryLength` and `TextLength` were heavily right-skewed, we applied logarithmic transformations ($\log_{1p}$ for summaries and $\log$ for text) to stabilize the variance. The resulting histograms (Figure 2) show a more symmetric distribution, making these features more suitable for linear modelling.

Cell 16 – Violin plots vs. score

```
import seaborn as sns

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.violinplot(x="Score", y="log_Summary", data=reviews)
plt.subplot(1, 2, 2)
sns.violinplot(x="Score", y="log_Text", data=reviews)
plt.show()
```

What it does:

Plots a violin plot of log summary length vs score, and log text length vs score.

Interpretation:

Shows how typical review length changes across scores (e.g., slightly longer texts for mid-range scores, etc.).

Report text:

Violin plots of log-transformed lengths stratified by score (Figure 3) reveal the relationship between review length and assigned rating, indicating modest differences in distribution across the five score levels.

9. Feature Matrices for Modeling

Cell 17 – Build `X_train` and `X_test`

```
X_train = pd.DataFrame(reduced, index=train.index,
                       columns=["PC {}".format(i) for i in range(1,
pca.n_components_ + 1)])
X_train["YrAfter2007"] = train["Time"].dt.year.map(lambda yr: max(yr -
2007, 0))
X_train["log_Summary"] = train["SummaryLength"].apply(np.log1p)
X_train["log_Text"] = train["TextLength"].apply(np.log)

test_text = test.apply(process_text, axis=1)
test_vectors = vectorizer.transform(test_text)
X_test = pd.DataFrame(pca.transform(test_vectors.toarray()),
index=test.index,
                       columns=["PC {}".format(i) for i in range(1,
pca.n_components_ + 1)])
X_test["YrAfter2007"] = test["Time"].dt.year.map(lambda yr: max(yr -
2007, 0))
X_test["log_Summary"] = test["SummaryLength"].apply(np.log1p)
X_test["log_Text"] = test["TextLength"].apply(np.log)
```

What it does:

For training set:

- Creates a DataFrame with all PCA components: PC 1, PC 2, ..., PC k.
- Adds:
 - `YrAfter2007`: 0 for reviews before 2007; `year - 2007` for reviews after 2007.

- `log_Summary`, `log_Text`: the log-transformed length features.

For **test set**:

- Applies same `process_text` → TF-IDF → PCA transformation.
- Adds the same `YrAfter2007`, `log_Summary`, `log_Text`.

Report text:

The final feature representation consisted of PCA-reduced TF-IDF components, a temporal feature indicating the number of years since 2007, and the log-transformed summary and text lengths. These features were constructed in the same manner for both training and testing sets.

10. Weighting Strategies for Training Samples

Cell 18 – Helpfulness-based weights

```
weights = train.apply(  
    lambda row: row["HelpfulnessNumerator"] ** row["HelpfulnessRatio"]  
    if row["HelpfulnessNumerator"] else 1,  
    axis=1  
)
```

(Exact exponent may differ, but concept is this.)

What it does:

- For reviews with at least one helpfulness vote:
 - Weight grows with both the numerator and the ratio (more helpful → higher weight).
- For reviews with 0 votes:
 - Weight = 1 (baseline).

Interpretation:

Gives **more influence** in training to reviews considered more helpful by users.

Cell 19 – Class-imbalance weights

```
score_freq = train["Score"].value_counts(normalize=True)
weights_alt = train["Score"].apply(lambda score: 1 /
score_freq[score])
```

What it does:

- Computes relative frequency of each score.
- Rare scores get larger weights; common scores get smaller weights.
(Classic inverse-frequency weighting.)

Report text:

Two weighting schemes were explored: one that up-weighted reviews with higher helpfulness scores, and another that inversely weighted samples according to score frequency to compensate for the class imbalance in the target variable.

11. Linear Regression Models

Helper: prediction post-processing

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
mean_squared_error

def predict_score(model, input_data):
    pred = model.predict(input_data)
    pred = np.round(pred).astype(int)
    pred = np.where(pred < 1, 1, pred)
    pred = np.where(pred > 5, 5, pred)
    return pred
```

What it does:

- Gets continuous predictions from a regression model.
 - Rounds to nearest integer.
 - Clips to range [1, 5], since valid scores are 1–5 only.
-

Model 1 – Weighted by helpfulness (Cell 20)

```
lr = LinearRegression()
lr.fit(X_train, train["Score"], sample_weight=weights)
lr_pred = predict_score(lr, X_test)
lr_cm = pd.DataFrame(confusion_matrix(test["Score"], lr_pred),
                     index=range(1, 6), columns=range(1, 6))

display(lr_cm)
print("Accuracy:", accuracy_score(test["Score"], lr_pred))
print("RMSE:", mean_squared_error(test["Score"], lr_pred,
squared=False))
```

Output:

- Confusion matrix (Table II in your report).
- Accuracy ≈ 0.4224 .
- RMSE ≈ 1.1008 .

Interpretation:

Model slightly underperforms compared to the best model later. Weighting by helpfulness does not help enough.

Model 2 – Weighted by inverse frequency (Cell 21)

```
lr2 = LinearRegression()
lr2.fit(X_train, train["Score"], sample_weight=weights_alt)
lr2_pred = predict_score(lr2, X_test)
```

```
lr2_cm = pd.DataFrame(confusion_matrix(test["Score"], lr2_pred),
                      index=range(1, 6), columns=range(1, 6))

display(lr2_cm)
print("Accuracy:", accuracy_score(test["Score"], lr2_pred))
print("RMSE:", mean_squared_error(test["Score"], lr2_pred,
squared=False))
```

Output:

- Confusion matrix (Table III).
- Accuracy ≈ 0.2317 .
- RMSE ≈ 1.3387 .

Interpretation:

- Much worse overall RMSE and accuracy.
- It tends to predict low scores better but fails badly on high scores.

Model 3 – Unweighted linear regression (Cell 22)

```
lr3 = LinearRegression()
lr3.fit(X_train, train["Score"])
lr3_pred = predict_score(lr3, X_test)
lr3_cm = pd.DataFrame(confusion_matrix(test["Score"], lr3_pred),
                      index=range(1, 6), columns=range(1, 6))

display(lr3_cm)
print("Accuracy:", accuracy_score(test["Score"], lr3_pred))
print("RMSE:", mean_squared_error(test["Score"], lr3_pred,
squared=False))
```

Output:

- Confusion matrix (Table IV).

- Accuracy ≈ 0.4262 .
- RMSE ≈ 1.0936 (best).

Interpretation:

- Surprisingly, the simplest model – unweighted linear regression – performs best.
- It is especially good at predicting high scores (4 and 5), which dominate the dataset.

Report text:

Three linear regression models were evaluated: one with helpfulness-based sample weights, one with inverse-frequency weights, and an unweighted baseline. The unweighted model achieved the best performance on the test set, with an accuracy of approximately 0.43 and an RMSE of 1.0936, outperforming both weighted variants.

12. Regularization: Lasso & Ridge

Cell 23 – Lasso sweep

```
from sklearn.linear_model import Lasso

alphas = np.linspace(0.000005, 0.0005, 100)
rmse = []
for alpha in alphas:
    lasso = Lasso(alpha=alpha)
    lasso.fit(X_train, train["Score"])
    lasso_pred = predict_score(lasso, X_test)
    rmse.append(mean_squared_error(test["Score"], lasso_pred,
    squared=False))

i = np.argmin(rmse)
plt.figure(figsize=(8, 6))
plt.plot(alphas, rmse)
plt.plot([alphas[i]], rmse[i], "ro")
plt.grid()
```

```
plt.title("RMSE vs alpha")
plt.show()
```

What it does:

- Sweeps 100 different values of `alpha`.
- For each `alpha`, fits a Lasso model, computes RMSE on test set.
- Plots RMSE vs `alpha` and marks the best one.

Observation:

Even the best Lasso result is worse than the unregularized linear regression (higher RMSE).

Cell 24 – Ridge sweep

```
from sklearn.linear_model import Ridge

alphas = np.linspace(0.000005, 0.0005, 100)
rmsees = []
for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train, train["Score"])
    ridge_pred = predict_score(ridge, X_test)
    rmsees.append(mean_squared_error(test["Score"], ridge_pred,
squared=False))

i = np.argmin(rmsees)
plt.figure(figsize=(8, 6))
plt.plot(alphas, rmsees)
plt.plot([alphas[i]], rmsees[i], "ro")
plt.grid()
plt.title("RMSE vs alpha")
plt.show()
```

What it does:

Same process as Lasso but using Ridge.

Observation:

Again, the best Ridge RMSE is worse than that of the simple linear model.

Report text:

To address potential overfitting, Lasso and Ridge regression models were trained over a range of regularization strengths. However, in both cases even the best-performing regularized models exhibited higher RMSE than the unregularized linear regression, suggesting that the feature set was already sufficiently constrained by PCA.

13. Final Submission

Cell 25 – Insert predictions

```
sample_submission["Predicted"] = lr3_pred
sample_submission
```

What it does:

- Adds a new column `Predicted` with the best model's predictions for each test review.

Cell 26 – Save CSV

```
sample_submission.to_csv("submission.csv")
```

What it does:

Writes `submission.csv` containing IDs and predicted scores → ready to submit to a competition platform or external evaluator.

Report text:

Finally, predictions generated by the unweighted linear regression model were inserted into the sample submission file and exported as `submission.csv` for evaluation.

How to Turn This into a Report Structure

You can now organize your report like this:

1. **Introduction** (dataset, goal, basic description).
2. **Data Preprocessing**
 - Time conversion
 - Missing summaries
 - HTML removal
 - Helpfulness correction
3. **Feature Engineering & Descriptive Analysis**
 - Summary/Text length, Helpfulness ratio
 - Descriptive statistics (Table 1)
 - Time series plots (Figure 1)
4. **Topic Modeling**
 - Bag-of-words, LDA topics (top words + interpretation)
 - Optional pyLDAvis mention
5. **Text Preprocessing & Feature Representation**
 - NLTK pipeline
 - TF-IDF (300 features)
 - PCA (80% variance)
 - Additional numeric features ([YrAfter2007](#), log lengths)
6. **Modeling and Evaluation**
 - Train/test split
 - Weighting strategies

- Three linear regression models (Tables II–IV)
- Comparison of accuracy and RMSE
- Lasso & Ridge experiments

7. Discussion & Conclusion

- Main insights
 - Limitations
 - Future work (e.g., non-linear models, deep learning, sentiment-aware models)
-
-

Summary and Future Work

In this work, we presented an end-to-end analysis and score prediction pipeline for the Amazon Fine Food Reviews dataset using **Python** as the primary programming language. The study began with extensive **data cleaning and preprocessing** of over 568,000 reviews, including timestamp conversion, handling of missing summaries, removal of HTML tags, and correction of inconsistent helpfulness values. We engineered key features such as **SummaryLength**, **TextLength**, and **HelpfulnessRatio**, along with log-transformed length features and a temporal feature measuring years after 2007, and examined their behavior through descriptive statistics, time-series plots, histograms, and violin plots.

For text understanding, we applied **text mining and NLP techniques** using **NLTK**, including tokenization, stopword removal, and lemmatization, and represented reviews via **TF-IDF** features (unigrams and bigrams). Dimensionality reduction was then performed using **Principal Component Analysis (PCA)** to retain 80% of the variance in a compact set of components. To uncover latent semantic structure, we trained a **Latent Dirichlet Allocation (LDA)** topic model (implemented with **scikit-learn** and visualized using **pyLDAvis**), which produced interpretable topics corresponding to distinct product categories such as dog food, beverages, snacks, coffee, and cooking ingredients.

On the predictive side, we constructed supervised models in **scikit-learn** using the PCA components and engineered numeric features as inputs. We evaluated several **regression-based approaches**:

- **Multiple Linear Regression** with helpfulness-based sample weights,

- **Multiple Linear Regression** with inverse-frequency class weights for score balancing,
- **Unweighted Multiple Linear Regression**,
- and regularized models including **Lasso Regression** and **Ridge Regression**.

Model performance was assessed using **confusion matrices**, **classification accuracy**, and **Root Mean Squared Error (RMSE)**. The best results were obtained with the **unweighted linear regression model**, which achieved an RMSE of approximately **1.0936** on the test set, outperforming all weighted and regularized variants. This indicates that, given the chosen feature set (TF-IDF + PCA + temporal and length features), a simple linear model is sufficient to capture much of the variance in review scores, while more aggressive weighting or coefficient penalization does not necessarily improve generalization.

The entire workflow was implemented in **Python** using a rich ecosystem of libraries and tools: **pandas** and **NumPy** for data manipulation, **matplotlib** and **seaborn** for visualization, **NLTK** for natural language processing, **scikit-learn** for TF-IDF, PCA, LDA, and regression models, and **pyLDAvis** for interactive topic visualization. In future work, this framework could be extended with more advanced **machine learning and deep learning methods**, such as tree-based ensembles (Random Forests, Gradient Boosting), **support vector machines**, or **neural architectures** (e.g., LSTMs, Transformers/BERT) to incorporate richer semantic information and potentially further reduce prediction error.

